

Corso di Laurea triennale in Ingegneria e Scienze Informatiche

**Analisi comparativa di modelli di Deep Learning per la
riduzione del rumore in immagini digitali**

Elaborato in: Metodi numerici per l'intelligenza artificiale

Relatore:

Prof. Lazzaro Damiana

Candidato:

Matteo Vanni

Matricola: 0000935584

Contents

Introduzione	i
METTI LE IMMAGINI	iii
1 Indice	1
1.1 Generale	1
1.1.1 Tipologie e Cause di Rumore nelle Immagini Digitali	1
1.1.2 Origini del rumore	2
1.1.3 Metriche di Valutazione	4
1.2 Denoising tradizionale	4
1.2.1 Filtri Spaziali	4
1.2.2 Denoising nel dominio della frequenza	4
1.3 Innovazione	5
1.3.1 Reti Convoluzionali (CNN)	5
1.3.2 Reti flessibili e adattive	5
1.3.3 Apprendimento auto-supervisionato	5
1.3.4 Prospettive future	5
1.4 Metodi esistenti	5
1.4.1 Approcci basati su patch	6
1.4.2 Metodi basati su sparse coding	6
1.4.3 Tecniche deep learning	6
1.5 Riflessione sulla letteratura	6
2 Reti neurali analizzate	9
2.1 RIDNet	9
2.1.1 Implementazione RIDNet	10
2.2 DnCNN	17
2.2.1 Implementazione DnCNN	18
2.3 Autoencoder	20
2.3.1 Implementazione Autoencoder	20
3 Dataset Utilizzati	23
3.1 BSD500	23
3.2 DIV2K	24
3.3 MIT-Adobe FiveK	24
3.4 SSID	25
3.5 RENOIR	25
3.6 Kvasir dataset	25

4	Elaborazione dei Dataset	27
4.0.1	Conversione immagini RAW	27
4.0.2	Classificazione immagini GT-Noisy	30
4.0.3	Duplicazione immagini	32
4.0.4	Padding di immagini con risoluzione inferiore a 512x512	35
4.1	Patching	36
4.2	Aggiunta del rumore	39
5	Analisi ed ottimizzazione	41
5.1	Prestazioni	41
5.2	Prima run	41
5.3	Quantizzazione dei modelli	41
5.3.1	Performance su dataset sconosciuti	41
6	Fine Tuning	43
7	Study	45

List of Figures

2.1	Architettura della rete: parte iniziale e finale. I blocchi intermedi sono omessi per semplicità.	15
-----	---	----

Listings

2.1	Implementazione del blocco di base BasicBlock	10
2.2	Implementazione del blocco di base con sigmoide BasicBlockSig	10
2.3	Blocco Merge_Run_dual per l'estrazione multiscala	11
2.4	Blocco ResidualBlock con connessione residua	11
2.5	Blocco EResidualBlock con convoluzioni a gruppi	12
2.6	Layer MeanShift per normalizzazione dei valori dei pixel	12
2.7	Layer di attenzione sui canali (CALayer)	13
2.8	Blocco completo che unisce moduli multiscala, residui e attenzione	13
2.9	Costruzione dell'architettura completa RIDNet	16
2.10	Corpo della ridnet	16
2.11	Ultimo blocco della rete	16
2.12	Pezzo finale della rete	17
2.13	Costruzione della RIDNet	17
2.14	Compilazione del modello	17
2.15	Definizione della funzione build_DnCNN	18
2.16	Primo strato convoluzionale	18
2.17	Ciclo con blocchi ripetuti	19
2.18	Strato finale e output	19
2.19	Definizione e primo livello convoluzionale	20
2.20	Secondo livello convolutivo + pooling	21
2.21	Terzo livello convolutivo + upsampling	21
2.22	Strato finale dell'autoencoder	21
4.1	Conversione immagini raw a jpg	28
4.2	Estensioni immagini raw	28
4.3	Scansione ricorsiva cartelle	28
4.4	Lettura immagini con OpenCV	28
4.5	Rinomina file	29
4.6	Salvataggio immagine	29
4.7	Caricamento delle immagini di RENOIR	30
4.8	Mean Squared Error	30
4.9	Classificazione batch con MSE	30
4.10	Calcolo di MSE	31
4.11	Elaborazione dei risultati	31
4.12	Salvataggio della classificazione	31
4.13	Funzione principale per la suddivisione e classificazione in batch	32
4.14	Definizione della funzione per processare la directory	32
4.15	Step 1: Gestione Reference	33
4.16	Step 2: Gestione Noisy	33
4.17	Step 3: Aggiunta del prefisso Batch_XXX_	34
4.18	Step 4: Copia nella cartella principale	34

4.19	Utilizzo della funzione <code>process_batch_folder</code>	34
4.20	Costanti con <code>direcotry</code> per BSD	35
4.21	Funzione di padding	35
4.22	Creazione del padding sui lati dell'immagine	35
4.23	Definizione funzione di elaborazione immagini	36
4.24	Costanti per il patching	37
4.25	Padding per il patching	37
4.26	Funzione di estrazione patch dalle immagini	37
4.27	Calcolo dei passi e creazione patch	38
4.28	Ciclo di elaborazione delle immagini e verifica preliminare	39
4.29	Aggiunta di rumore da lista	39
4.30	Aggiunta del rumore da funzione	39

Introduzione

In questa tesi progettuale si vuole analizzare in dettaglio le architetture di varie reti neurali, confrontandone prestazioni, e applicazioni. Si andrà a spiegare cosa significa Rumore in immagini e cosa comporta e come compare,

Qui si rimuove rumore alle immagini con studio di modelli allenati con cose diverse da immagini mediche e poi messi a lavorare su immagini mediche, poi boh gli si fa du merende

METTI LE IMMAGINI

1 Indice

1.1 Generale

Il denoising delle immagini rappresenta una problematica centrale nell'ambito dell'elaborazione digitale delle immagini. Si tratta di un processo volto a rimuovere il rumore che si introduce durante la cattura, la trasmissione o la memorizzazione di un'immagine digitale, con l'obiettivo di recuperare una rappresentazione più fedele e pulita del contenuto visivo originale.

La presenza di rumore può compromettere significativamente la qualità visiva e l'accuratezza di analisi successive, come il riconoscimento di oggetti o la segmentazione. Formalmente, il problema può essere modellato come:

$$y = x + n$$

dove y è l'immagine osservata contenente rumore, x è l'immagine pulita da recuperare e n rappresenta il rumore additivo.

1.1.1 Tipologie e Cause di Rumore nelle Immagini Digitali

Il rumore presente nelle immagini può assumere diverse forme, a seconda della sorgente e delle condizioni di acquisizione. I principali modelli e le relative cause sono:

- **Rumore gaussiano additivo (AWGN):** spesso usato come modello ideale per rumori sommatoria di più sorgenti e per molte analisi teoriche. Nelle immagini acquisite, una componente a distribuzione approssimativamente gaussiana deriva da disturbi elettronici del circuito di acquisizione (read noise, thermal noise). Per il rumore reale dei sensori si usa frequentemente il modello Poisson–Gaussiano (shot noise + rumore elettronico) e metodi specifici per la sua stima. (Janesick 2007; Foi et al. 2008)
- **Rumore di tipo Poisson (shot noise):** dovuto alla natura discreta dei fotoni rilevati dal sensore; la varianza è proporzionale al segnale stesso. Tecniche come la trasformazione di Anscombe e i suoi sviluppi (e.g. inversioni ottimali) sono largamente usate per stabilizzare la varianza prima del denoising. (Mäkitalo and Foi 2013; Foi et al. 2008)
- **Rumore impulsivo (“salt-and-pepper”):** si manifesta come singoli pixel con valori estremi (0 o massimo) e tipicamente è causato da errori di trasmissione, cellule pixel difettose o errori A/D; esistono filtri switch, mediani adattivi e approcci più recenti (SVM, reti profonde) dedicati alla sua rimozione. (Gómez-Moreno et al. 2014)
- **Rumore speckle (moltiplicativo):** caratteristico dei sistemi coerenti (SAR, ultrasuoni, laser), ha natura moltiplicativa e statistiche particolari; richiede metodi di despeckling specifici che tengano conto della natura non additiva del disturbo. (**despeckle’survey**; Goodman 1976)

- **Artefatti di compressione (lossy):** la compressione con perdita (e.g. JPEG) introduce blocchi, ringing e perdita di dettaglio ad alta frequenza; questi disturbi non sono “rumore” stocastico puro ma effetti deterministici dell’analisi quantizzazione/trasformata che richiedono correzione mirata. (Wallace 1991; Eschbach 1997)

1.1.2 Origini del rumore

Le principali origini fisiche e operative del rumore nelle immagini sono:

- **Fenomeni fotonici e statistica di conteggio (shot noise):** il processo di conteggio fotonico genera rumore Poissoniano la cui intensità dipende dal livello di illuminazione; nei sistemi a basso conteggio questo è dominante. (Foi et al. 2008; Janesick 2007)
- **Rumore elettronico del sensore (read noise, dark current, quantizzazione):** deriva dai circuiti di lettura (amplificatori, ADC), dal dark current termico e da imperfezioni di fabbricazione; la loro combinazione spesso giustifica il modello Poisson-Gaussiano per i dati raw. (Janesick 2007; Foi et al. 2008)
- **Errori di trasmissione e memorizzazione:** perdite o bit-flip durante la trasmissione e settori di memoria corrotti possono introdurre rumore impulsivo o artefatti localizzati (salt-and-pepper). (Gómez-Moreno et al. 2014)
- **Interferenze ambientali e condizioni di acquisizione:** scarsa illuminazione, elevata temperatura, luce dispersa e vibrazioni incrementano il rumore e possono cambiare la sua natura (es. aumento di dark current o rumore termico). (Janesick 2007)
- **Processi di compressione e post-elaborazione:** quantizzazione e compressione con perdita introducono artefatti strutturati (blocchi, ringing) che spesso vengono trattati con tecniche di post-processing specifiche, non con semplici filtri statistici. (Wallace 1991; Eschbach 1997)

Metodi classici

I primi approcci si basavano su filtri spaziali:

- Filtri lineari come il filtro medio e il filtro gaussiano, che agiscono mediando i valori dei pixel in un intorno locale. Sebbene semplici, tendono a sfocare i dettagli fini dell’immagine.
- Filtri non lineari come il filtro mediano, efficace nel rimuovere rumore impulsivo (è dimostrato che mantiene i bordi meglio dei filtri lineari per rumore moderato (**arias-castro2009**; Arias-Castro and Donoho 2009)), e il filtro bilaterale, che preserva i bordi mediante un’azione locale-spaziale e di intensità (Tomasi and Manduchi 1998; Paris et al. 2008).

Miglioramenti ulteriori si sono ottenuti con la trasformata wavelet: Donoho e Johnstone hanno introdotto il denoising via soglia nei coefficienti wavelet (“wavelet shrinkage”) (Donoho 1995), che costituisce una pietra miliare. Ulteriori lavori ne hanno studiato variazioni e parametri di soglia ottimali (Chambolle et al. 2003; Denoising 2020).

Modelli Statistici e Filtri Non Locali

Un progresso significativo è stato portato dai modelli statistici e dai filtri basati sulla similarità delle patch:

- **Markov Random Fields (MRF):** modelli probabilistici che catturano la dipendenza tra pixel vicini.
- **Non-Local Means (NLM)** (Buades, Coll, and Morel 2005): sfrutta la ridondanza strutturale delle immagini, mediando patch simili non necessariamente vicine.
- **BM3D (Block Matching 3D)** (Dabov et al. 2007): raggruppa patch simili in domini tridimensionali ed applica trasformate collaborative, con risultati allo stato dell'arte per lungo tempo.

Sparse Coding e Dictionary Learning

La rappresentazione sparsa ha introdotto un paradigma potente:

- **Sparse Coding:** consente di discriminare il segnale dal rumore ricercando rappresentazioni con pochi coefficienti non nulli.
- **K-SVD:** algoritmo iterativo per apprendere un dizionario ottimale dalle patch di immagine, migliorando la capacità di ricostruzione.

Sparse Coding e Dictionary Learning

La rappresentazione sparsa ha introdotto un paradigma potente per il denoising delle immagini, fondato sull'assunzione che le patch possano essere rappresentate come combinazioni lineari di pochi atomi selezionati da un dizionario (over-complete). Il modello standard si esprime attraverso:

$$\hat{\alpha} = \arg \min_{\alpha} \|y - D\alpha\|_2^2 + \lambda \|\alpha\|_1$$

dove D è il dizionario e α il vettore dei coefficienti sparsi (Fan et al. 2019; H. Wang et al. 2020a).

- **K-SVD:** algoritmo iterativo che alterna sparse coding e aggiornamento di atomi attraverso SVD, apprendendo un dizionario ottimale per il denoising (Lebrun and Leclaire 2012; Wikipedia contributors 2025).
- **Migliorie recenti:** versioni avanzate come il $K\text{-SVD}_P$, che applica algoritmi primal-dual per migliorare efficacia e velocità con rumore elevato (Xiao et al. 2020).
- **Approcci basati su rete neurale:** reti come SRNet integrano il framework di sparse coding in reti CNN multiscala, accelerando l'ottimizzazione e migliorando la qualità, soprattutto nei dettagli testurizzati (Sheng et al. 2022).
- **Sparse coding convoluzionale:** estende il modello a dizionari convoluzionali adatti a strutture locali ripetute; ad oggi è all'origine di architetture on-line più efficienti (Garcia-Cardona and Wohlberg 2018; Y. Zhang et al. 2022).
- **Denoising cooperativo distribuito:** proposte come *Cloud K-SVD* permettono l'apprendimento di dizionari condivisi tra nodi, preservando qualità e generalizzazione (Christian Marius Lillelund, Henrik Bagger Jensen, and Christian Fischer Pedersen 2022).

Metodi Basati su Apprendimento Automatico

L'avvento del machine learning ha rivoluzionato il denoising:

- **Reti neurali convoluzionali (CNN):** modelli come DnCNN (K. Zhang, Zuo, Chen, et al. 2017) e FFDNet (K. Zhang, Zuo, and L. Zhang 2018) hanno mostrato notevoli miglioramenti nella rimozione del rumore.
- **Metodi auto-supervisionati:** Noise2Noise (Lehtinen et al. 2018) permette di addestrare modelli senza immagini pulite di riferimento.
- **Real Image Denoising:** metodi come RIDNet (Anwar and Barnes 2019) introducono meccanismi di attenzione per gestire meglio il rumore reale catturato dalle fotocamere.

Più recentemente, modelli basati su *transformer* hanno ampliato la capacità di catturare relazioni non locali e migliorato la generalizzazione.

1.1.3 Metriche di Valutazione

La valutazione dei metodi di denoising si basa su:

- **PSNR (Peak Signal-to-Noise Ratio):** misura il rapporto tra segnale massimo ed errore quadratico medio (Hore and Ziou 2010).
- **SSIM (Structural Similarity Index):** valuta la similarità strutturale percepita, considerando luminanza, contrasto e struttura (Z. Wang et al. 2004).

La valutazione qualitativa visiva rimane cruciale in contesti come quello medico o artistico (Buades, Coll, and Morel 2005; Dabov et al. 2007).

1.2 Denoising tradizionale

I metodi tradizionali si basano principalmente su filtri spaziali e trasformati nel dominio della frequenza (Gonzalez and Woods 2018).

1.2.1 Filtri Spaziali

- **Filtro medio:** sostituisce ogni pixel con la media dei pixel circostanti, semplice ma tende a sfocare i bordi (Gonzalez and Woods 2018).
- **Filtro gaussiano:** convoluzione con kernel gaussiano pesato, preserva meglio i bordi (Gonzalez and Woods 2018).
- **Filtro mediano:** robusto al rumore impulsivo, efficace soprattutto con rumore sale e pepe (T. S. Huang and Tang 1979).

1.2.2 Denoising nel dominio della frequenza

Mediante la trasformata di Fourier è possibile separare le componenti ad alta frequenza (spesso associate al rumore) da quelle a bassa frequenza (contenuto strutturale) (Oppenheim and Schaffer 1999). Questo approccio è utile in presenza di rumore periodico, ma tende a sacrificare dettagli fini.

Limiti principali:

- Scarsa adattabilità al contenuto dell'immagine.
- Compromesso tra riduzione del rumore e perdita di dettaglio.
- Prestazioni limitate su rumore non uniforme o strutturato.

1.3 Innovazione

Negli ultimi anni, l'intelligenza artificiale ha rivoluzionato il denoising delle immagini. L'integrazione di reti profonde e metodi auto-supervisionati ha permesso di superare le limitazioni dei metodi tradizionali, ottenendo prestazioni superiori su dataset standard come BSD500 (Martin et al. 2001), MIT-Adobe FiveK (Bychkovsky et al. 2011), SIDD (Abdelhamed, Timofte, Brown, et al. 2019), DIV2K (Ignatov, Timofte, et al. 2019) e RENOIR (Anaya and Barbu 2014).

1.3.1 Reti Convoluzionali (CNN)

Le CNN sono state tra le prime architetture a essere impiegate per il denoising. DnCNN (K. Zhang, Zuo, Chen, et al. 2017) è un esempio di successo, che ha introdotto il concetto di residual learning, permettendo alla rete di concentrare l'attenzione sulle componenti rumorose.

1.3.2 Reti flessibili e adattive

Modelli come FFDNet (K. Zhang, Zuo, and L. Zhang 2018) hanno introdotto la possibilità di gestire livelli variabili di rumore in input, migliorando l'efficienza e la generalizzazione. Altri modelli hanno iniziato a incorporare meccanismi di attenzione per focalizzarsi selettivamente sulle aree più degradate.

1.3.3 Apprendimento auto-supervisionato

L'approccio *Noise2Noise* (Lehtinen et al. 2018) ha dimostrato che è possibile allenare un modello a ripulire immagini anche senza avere accesso all'immagine pulita di riferimento, ma solo a immagini rumorose multiple della stessa scena.

1.3.4 Prospettive future

L'introduzione di modelli basati su Transformer e reti neurali generative (GAN) promette di migliorare ulteriormente la qualità del denoising, specialmente in scenari complessi, come immagini mediche, notturne o compresse.

1.4 Metodi esistenti

La letteratura sul denoising delle immagini è estremamente ricca e articolata. Possiamo suddividere i metodi esistenti in tre macro-categorie: approcci tradizionali, metodi basati su patch, e tecniche di deep learning.

1.4.1 Approcci basati su patch

Metodi come **Non-Local Means** (NLM) (Buades, Coll, and Morel 2005) e **BM3D** (Dabov et al. 2007) si basano sul concetto di similitudine tra porzioni (patch) di immagine. Questi algoritmi ricercano blocchi simili all'interno dell'immagine per effettuare una media ponderata più efficace. BM3D, in particolare, è considerato uno dei migliori metodi non-appresi tuttora in uso.

1.4.2 Metodi basati su sparse coding

Questi approcci modellano ogni patch come una combinazione sparsa di elementi di un dizionario. Algoritmi come K-SVD (Aharon, Elad, and Bruckstein 2006; H. Wang et al. 2020a; Christian M. Lillielund, Henrik B. Jensen, and Christian F. Pedersen 2022) apprendono un dizionario adattivo dai dati per una rappresentazione più efficiente. Sebbene più costosi computazionalmente, offrono buoni risultati su immagini naturali e sono particolarmente efficaci nel preservare dettagli locali rispetto ai filtri classici.

La rappresentazione sparsa ha rappresentato un passo fondamentale nell'evoluzione dei metodi di denoising, introducendo concetti di apprendimento adattivo e modellazione statistica dei dati, aprendo la strada ai successivi approcci basati su reti neurali.

1.4.3 Tecniche deep learning

Dalla metà degli anni 2010, l'adozione di modelli convoluzionali ha permesso un salto di qualità significativo. Reti come DnCNN (K. Zhang, Zuo, Chen, et al. 2017) e FFDNet (K. Zhang, Zuo, and L. Zhang 2018) hanno dimostrato come reti profonde possano apprendere direttamente la mappatura da immagini rumorose a pulite. Approcci auto-supervisionati come Noise2Noise (Lehtinen et al. 2018) hanno reso possibile l'addestramento senza immagini pulite di riferimento.

Modelli più recenti basati su U-Net, GAN e transformer (Anwar and Barnes 2019) hanno ulteriormente migliorato la capacità di generalizzazione a diversi tipi di rumore e scenari complessi. L'evoluzione dal denoising classico a modelli di deep learning mostra come l'integrazione di apprendimento adattivo e capacità di modellare relazioni non locali abbia trasformato il campo.

1.5 Riflessione sulla letteratura

Nonostante i risultati eccellenti dei modelli deep learning, permangono sfide aperte:

- L'adattamento a rumori complessi o misti che non corrispondono ai modelli sintetici utilizzati in fase di training (H. Wang et al. 2020a; H. Wang et al. 2020b).
- L'addestramento in assenza di dati puliti o con dataset limitati.
- L'applicazione in tempo reale per video e flussi di immagini, dove la complessità computazionale dei modelli può essere un limite.
- L'integrazione di metodi di denoising con altri task di miglioramento delle immagini, come super-risoluzione o deblurring, rappresenta una direzione promettente.

L'analisi della letteratura evidenzia come il denoising delle immagini abbia conosciuto un'evoluzione significativa: dai filtri spaziali e trasformate classiche ai complessi modelli neurali.

Ogni approccio presenta vantaggi e limiti specifici, e le ricerche più recenti tendono a combinare metodi classici e deep learning, oppure ad incorporare conoscenza specifica del dominio, per ottenere soluzioni più robuste e generalizzabili.

2 Reti neurali analizzate

In questo capitolo vengono descritte le principali architetture di reti neurali utilizzate per il problema della rimozione del rumore dalle immagini (*image denoising*). La scelta delle reti è stata effettuata in base alla loro diffusione nella letteratura scientifica, alla semplicità di implementazione e alla capacità di rappresentare approcci diversi allo stesso problema.

Le tre architetture analizzate sono:

- **RIDNet** – una rete moderna e avanzata, specificamente progettata per il denoising di immagini reali;
- **DnCNN** – un modello pionieristico che ha introdotto l'idea del denoising come stima del residuo;
- **Autoencoder** – un'architettura generica, versatile e adattabile a diversi scenari di apprendimento non supervisionato.

2.1 RIDNet

La **Residual Image Denoising Network** (RIDNet) rappresenta uno dei modelli più moderni per la riduzione del rumore in immagini naturali. Il suo principale punto di forza è la capacità di gestire rumori reali, come quelli provenienti dai sensori fotografici digitali, che non seguono distribuzioni statistiche ideali. L'architettura è basata su *blocchi residui* e meccanismi di attenzione che consentono di bilanciare in maniera ottimale la riduzione del rumore e la preservazione dei dettagli fini.

In particolare:

- l'uso di **connessioni residue** facilita il flusso dell'informazione, riducendo la perdita di dettaglio e migliorando la stabilità dell'addestramento;
- l'integrazione di **attenzione spaziale e di canale** consente di focalizzare l'elaborazione sulle regioni e sulle caratteristiche più rilevanti per il denoising;
- la progettazione modulare rende RIDNet scalabile e adattabile a immagini di diversa risoluzione e qualità.

Grazie a queste caratteristiche, RIDNet è considerata uno standard di riferimento per il denoising in scenari realistici, superando molti dei limiti dei metodi classici basati su rumori sintetici.

2.1.1 Implementazione RIDNet

In questa sezione viene riportata l'implementazione della rete **RIDNet** in Keras/TensorFlow presa dal caso di studio **Real Image Denoising with Feature Attention** (Anwar and Barnes 2019)

Blocchi di base

Listing 2.1: Implementazione del blocco di base BasicBlock

```

1 @register_keras_serializable( BasicBlock )
2 def BasicBlock(x, out_channels, ksize=3, stride=1, pad='same'):
3     x = Conv2D(out_channels, ksize, strides=stride, padding=pad)(
4         ↪ x)
5     x = ReLU()(x)
6     return x

```

Questo blocco esegue due operazioni fondamentali:

1. Una **convoluzione**, cioè un'operazione che “legge” l'immagine con un filtro (o kernel) e produce nuove mappe di caratteristiche (*feature maps*). Queste mappe contengono informazioni sui bordi, sui contorni e su pattern locali.
2. Una funzione di attivazione **ReLU**, che sostituisce i valori negativi con zero e mantiene i positivi invariati. Questo passaggio rende la rete capace di apprendere relazioni non lineari, fondamentali per rappresentare dettagli complessi dell'immagine.

In altre parole, il **BasicBlock** prende un'immagine o una mappa di caratteristiche e ne produce una nuova versione, mettendo in evidenza strutture locali utili per distinguere rumore da dettaglio.

Listing 2.2: Implementazione del blocco di base con sigmoide BasicBlockSig

```

1 @register_keras_serializable( BasicBlockSig )
2 def BasicBlockSig(x, out_channels, ksize=3, stride=1, pad='same')
3     ↪ :
4     x = Conv2D(out_channels, ksize, strides=stride, padding=pad)(
5         ↪ x)
6     x = tf.keras.activations.sigmoid(x)
7     return x

```

La differenza con il blocco precedente è che al posto della **ReLU** utilizza una **sigmoide**.

La sigmoide restituisce valori compresi tra 0 e 1, ed è molto utile quando vogliamo “pesare” le informazioni: un valore vicino a 1 indica che una certa caratteristica è molto importante, mentre un valore vicino a 0 indica che può essere trascurata.

Questo blocco viene usato nei moduli di attenzione.

Blocchi avanzati

Listing 2.3: Blocco Merge_Run_dual per l'estrazione multiscala

```

1 @register_keras_serializable( Merge_Run_dual )
2 def Merge_Run_dual(x, in_channels, out_channels):
3     b1 = Conv2D(out_channels, 3, padding='same')(x)
4     b1 = ReLU()(b1)
5     b1 = Conv2D(out_channels, 3, padding='same', dilation_rate=2)
6         ↪ (x)
7     b1 = ReLU()(b1)
8     b2 = Conv2D(out_channels, 3, padding='same', dilation_rate=3)
9         ↪ (x)
10    b2 = ReLU()(b2)
11    b2 = Conv2D(out_channels, 3, padding='same', dilation_rate=4)
12        ↪ (x)
13    b2 = ReLU()(b2)
14    c = Concatenate(axis=-1)([b1, b2])
15    out = Conv2D(out_channels, 3, padding='same')(c)
16    out = ReLU()(out)
17    return Add()([out, x])

```

- L'input viene analizzato con convoluzioni che hanno diversi **tassi di dilatazione**. La dilatazione permette al filtro di “saltare” dei pixel, e quindi di catturare informazioni non solo dai pixel vicini, ma anche da quelli più lontani. In questo modo, il modello riesce a vedere sia dettagli fini sia strutture più ampie.
- I risultati vengono **concatenati**, cioè messi uno accanto all'altro, e passati a un'altra convoluzione che li mescola.
- Infine, si aggiunge l'output all'input originale (**connessione residua**), in modo da non perdere informazioni.

Questo blocco serve a dare al modello la capacità di comprendere l'immagine a più “scale” contemporaneamente.

Listing 2.4: Blocco ResidualBlock con connessione residua

```

1 @register_keras_serializable( ResidualBlock )
2 def ResidualBlock(x, out_channels):
3     res = Conv2D(out_channels, 3, padding='same')(x)
4     res = ReLU()(res)
5     res = Conv2D(out_channels, 3, padding='same')(res)
6     return ReLU()(Add()([res, x]))

```

Questo è un classico **blocco residuo**, introdotto con le ResNet(Reti Neurali Residuali). L'idea è che invece di trasformare completamente l'input, la rete impara a calcolare solo la “correzione” necessaria (residuo). Questo rende l'apprendimento più stabile e veloce. Il risultato è la somma tra input originale e la trasformazione appresa.

Listing 2.5: Blocco EResidualBlock con convoluzioni a gruppi

```

1 @register_keras_serializable( EResidualBlock )
2 def EResidualBlock(x, out_channels, group=1):
3     res = Conv2D(out_channels, 3, padding='same', groups=group)(x
4         ↪ )
5     res = ReLU()(res)
6     res = Conv2D(out_channels, 3, padding='same', groups=group)(
7         ↪ res)
8     res = ReLU()(res)
9     res = Conv2D(out_channels, 1, padding='valid')(res)
10    return ReLU()(Add()([res, x]))

```

Variante più profonda del blocco residuo. Qui vengono usate le **convoluzioni a gruppi**, che dividono i canali in sottoinsiemi elaborati separatamente: in questo modo il modello può specializzarsi su diversi tipi di caratteristiche senza aumentare troppo i calcoli.

Alla fine una convoluzione 1x1 (Conv2D(out_channels, 1, [...])) ricompone i canali.

Layer di supporto

Listing 2.6: Layer MeanShift per normalizzazione dei valori dei pixel

```

1 @register_keras_serializable( MeanShift )
2 def MeanShift(rgb_range, rgb_mean, rgb_std, sign=-1):
3     @register_keras_serializable( shift )
4     def shift(x):
5         rgb_mean_tensor = tf.constant(rgb_mean, dtype=tf.float32)
6         rgb_std_tensor = tf.constant(rgb_std, dtype=tf.float32)
7         return (x + sign * rgb_range * rgb_mean_tensor) /
8             ↪ rgb_std_tensor
9     shift_layer = Lambda(shift)
10    shift_layer.shift_fn = shift
11    return shift_layer

```

Serve a **normalizzare** i valori dei pixel. Le immagini hanno tipicamente valori compresi tra 0 e 255, ma è più facile per la rete lavorare su dati con media zero e varianza unitaria.

Questo strato sposta (shift(x)) e ridimensiona i valori dei pixel in base alla media e deviazione standard di ciascun canale RGB.

Listing 2.7: Layer di attenzione sui canali (CALayer)

```

1 @register_keras_serializable( CALayer )
2 def CALayer(x, channel, reduction=16):
3     y = GlobalAveragePooling2D()(x)
4     y = Reshape((1, 1, channel))(y)
5     y = BasicBlock(y, channel // reduction, ksize=1, pad='valid')
6     y = BasicBlockSig(y, channel, ksize=1, pad='valid')
7     return Multiply()([x, y])

```

È un **layer di attenzione sui canali**. Funziona così:

1. Fa una media globale per ogni canale (es. canale rosso, verde, blu), in modo da capire quanto ciascun canale contribuisce all'immagine(`GlobalAveragePooling2D`).
2. Passa questi valori attraverso piccole reti (convoluzioni 1x1 + sigmoide) per ottenere dei pesi tra 0 e 1.
3. Moltiplica questi pesi ai canali originali: accendendo i canali più utili e spegnendo quelli meno importanti(`Multiply()([x, y])`).

Blocco completo

Alla fine i layer `Merge_Run_Dual`2.3, `ResidualBlock`2.4, `EResidualBlock`2.5 e `CALayer`2.7 sono riuniti a formare l'unità base della rete

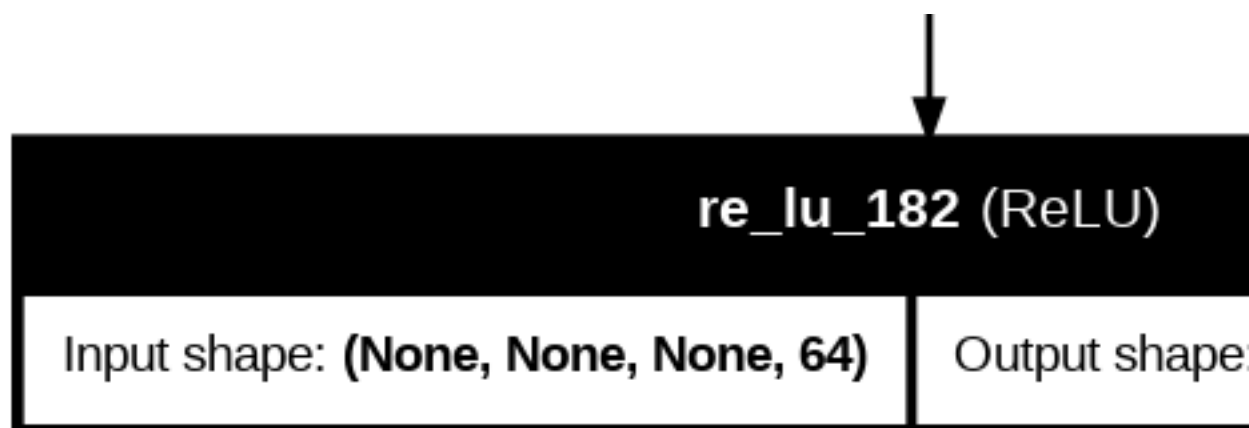
Listing 2.8: Blocco completo che unisce moduli multiscala, residui e attenzione

```

1 @register_keras_serializable( Block )
2 def Block(x, in_channels, out_channels):
3     x = Merge_Run_dual(x, in_channels, out_channels)
4     x = ResidualBlock(x, out_channels)
5     x = EResidualBlock(x, out_channels)
6     x = CALayer(x, out_channels)
7     return x

```


Architettura RIDNet



⋮
... blocchi intermedi omessi ...

Listing 2.9: Costruzione dell'architettura completa RIDNet

```

1 @register_keras_serializable( build_RIDNET )
2 def build_RIDNET(input_shape, n_feats, reduction, rgb_range=1.0):
3     rgb_mean = (0.4488, 0.4371, 0.4040)
4     rgb_std = (1.0, 1.0, 1.0)
5
6     inputs = Input(shape=input_shape)
7     x = MeanShift(rgb_range, rgb_mean, rgb_std, sign=-1)(inputs)

```

In questa prima parte viene definita la funzione `build_RIDNET`, che costruisce il modello.

- L'input è un'immagine con una forma specificata da `input_shape`.
- Prima di elaborarla, viene applicato un `MeanShift`, che normalizza i valori dei pixel in base alla media e alla deviazione standard dei canali RGB.

Questo passaggio serve a semplificare l'apprendimento della rete.

Listing 2.10: Corpo della ridnet

```

1 build_RIDNET
2     x = BasicBlock(x, n_feats)
3     b1 = Block(x, n_feats, n_feats)
4     b2 = Block(b1, n_feats, n_feats)
5     b3 = Block(b2, n_feats, n_feats)
6     b4 = Block(b3, n_feats, n_feats)

```

Nella parte centrale:

- Viene applicato un primo `BasicBlock`, che funziona come livello iniziale di estrazione delle caratteristiche.
- Successivamente, l'immagine passa attraverso una serie di `Block`, ognuno composto da più sottostrutture (convoluzioni multiscala, blocchi residui e meccanismi di attenzione sui canali).
- I blocchi sono concatenati (`b1`, `b2`, `b3`, `b4`), così che la rete possa catturare informazioni via via più complesse e dettagliate. La struttura dell'esperimento citato (Anwar and Barnes 2019) usa solo quattro blocchi

Listing 2.11: Ultimo blocco della rete

```

1 build_RIDNET
2     res = Conv2D(3, 3, padding='same')(b4)

```

Dopo l'ultimo blocco, si applica una convoluzione finale con 3 filtri, corrispondenti ai tre canali RGB.

Il risultato è un'immagine "ricostruita" che rappresenta una prima versione denoised dell'input.

Listing 2.12: Pezzo finale della rete

build_RIDNET

```

1 out = MeanShift(rgb_range, rgb_mean, rgb_std, sign=1)(res)
2 out = Add()([out, inputs])
3 model = Model(inputs=inputs, outputs=out)
4 return model

```

Infine:

- Si applica nuovamente un **MeanShift**, questa volta con **sign=1**, per riportare i valori dei pixel alla scala originale.
- Viene effettuata una somma residua (**Add**) tra l'output elaborato e l'immagine di input: questo permette alla rete di concentrarsi sulla rimozione del rumore, mantenendo però i dettagli originali.
- Il modello finale viene costruito e restituito.

In questo modo, la **RIDNet** sfrutta il concetto di connessioni residue e blocchi multiscala per ridurre il rumore dalle immagini mantenendo il più possibile la loro qualità visiva.

Listing 2.13: Costruzione della RIDNet

```

1 model = build_RIDNET(input_shape=(None, None, 3), n_feats=64,
    ↪ reduction=16)

```

Dunque la rete vien così inizializzata, seguendo in parte l'esperimento di partenza utilizzando lo shape delle immagini di input come **None** per poter analizzare qualsiasi tipo di risoluzione ma lasciando 3 per specificare immagini a colori, 64 features map (filtri convoluzionali interni) e la reduction a 16 che serve nei blocchi di attenzione per comprimere temporaneamente le informazioni e poi espanderle, riducendo il costo computazionale.

Listing 2.14: Compilazione del modello

```

1 model.compile(optimizer=tf.keras.optimizers.Adam(1e-04), loss=
    ↪ mse, metrics=[ accuracy ])

```

Viene compilata con:

- Come ottimizzatore Adam con tasso di apprendimento $1e^{-4}$ utilizzato per aggiornare i pesi della rete in maniera stabile ed efficace.
- Come funzione di errore la scelta è Mean Squared Error (MSE), cioè l'errore quadratico medio, tipico nei problemi di denoising e ricostruzione di immagini.
- infine viene aggiunta l'accuratezza del modello nelle metriche di valutazione per poter monitorare l'andamento del modello e poterne fare grafici

2.2 DnCNN

La **Denoising Convolutional Neural Network** (DnCNN) è una rete convoluzionale profonda introdotta per la prima volta come architettura dedicata al denoising. La sua importanza è legata all'idea innovativa di formulare il problema non come una mappatura diretta tra immagine rumorosa e immagine pulita, ma come la stima del **residuo** (ossia il rumore stesso).

Il funzionamento di DnCNN può essere riassunto in tre fasi principali:

1. riceve in input un'immagine rumorosa;
2. apprende a predire la componente di rumore associata a quell'immagine;
3. sottrae il rumore stimato dall'immagine originale, ottenendo un risultato denoised.

L'architettura utilizza più strati convoluzionali seguiti da *batch normalization* e funzioni di attivazione ReLU, che permettono di estrarre rappresentazioni sempre più complesse. Uno dei vantaggi di DnCNN è la sua capacità di generalizzare a diversi livelli di rumore, senza dover essere riaddestrata per ogni condizione.

Per questi motivi, DnCNN viene spesso considerata un punto di riferimento sperimentale e un'architettura di base su cui sviluppare metodi più sofisticati.

2.2.1 Implementazione DnCNN

Qui di seguito l'implementazione iniziale della rete seguendo l'esperimento **Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising** (K. Zhang, Zuo, Chen, et al. 2017)

Definizione della funzione principale

Listing 2.15: Definizione della funzione build_DnCNN

```
1 def build_DnCNN(input_shape):
2     input = Input(input_shape, name = 'input')
```

Definizione della funzione Qui viene definita la funzione `build_DnCNN`, che riceve come parametro la dimensione dell'immagine in ingresso (`input_shape`). Viene quindi creato il *layer di input*, ossia il punto da cui i dati entreranno nella rete.

Primo strato convoluzionale

Listing 2.16: Primo strato convoluzionale

```
1 x = Conv2D(64, kernel_size = (3,3), padding = 'same', name =
   ↪ 'conv2d_l1')(input)
2 x = Activation('relu', name = 'act_l1')(x)
```

Primo strato Viene applicata una convoluzione con 64 filtri 3×3 con padding `same`, seguita da una funzione di attivazione ReLU. Questo consente alla rete di estrarre le prime caratteristiche di basso livello dall'immagine di input.

Blocchi ripetuti

Listing 2.17: Ciclo con blocchi ripetuti

```

1  for i in range(17):
2      x = Conv2D(64, kernel_size = (3,3), padding = 'same',
      ↪ name = 'conv2d_' + str(i))(x)
3      x = BatchNormalization(axis = -1, name = 'BN_' + str(i))(
      ↪ x)
4      x = Activation('relu', name = 'act_' + str(i))(x)

```

Blocchi intermedi La rete esegue un ciclo di 17 blocchi identici, ognuno composto da:

- una convoluzione con 64 filtri
- una normalizzazione batch (`BatchNormalization`) per stabilizzare e velocizzare l'allenamento
- una funzione di attivazione `ReLU` per introdurre non-linearità

Questi blocchi permettono di catturare informazioni più profonde e complesse, rendendo la rete capace di modellare il rumore presente nell'immagine.

Strato finale e sottrazione residua

Listing 2.18: Strato finale e output

```

1  x = Conv2D(3, kernel_size = (3,3), padding = 'same', name = '
      ↪ conv2d_13')(x)
2  x = Subtract(name = 'subtract')([input, x])
3  return Model(input, x)

```

Alla fine, un ultimo strato convoluzionale riduce i canali di uscita a 3 (uno per ogni canale RGB). Successivamente, viene eseguita una sottrazione tra l'immagine di input e il risultato della convoluzione.

Questa operazione implementa il concetto di **residual learning**: la rete impara a stimare il rumore presente nell'immagine, che viene poi sottratto per ottenere un'immagine pulita.

2.3 Autoencoder

Gli **autoencoder** costituiscono un'architettura più generica, non progettata specificamente per il denoising ma ampiamente utilizzata a tale scopo grazie alla loro natura di rete compressiva e ricostruttiva. Un autoencoder è composto da due blocchi principali:

- un **encoder**, che riduce progressivamente la dimensione dell'immagine di input, comprimendola in uno spazio latente a bassa dimensionalità;
- un **decoder**, che tenta di ricostruire l'immagine originale a partire dalla rappresentazione latente.

Nel contesto del denoising, l'autoencoder sfrutta il fatto che il processo di compressione tende a eliminare le variazioni casuali non rilevanti (come il rumore), mentre il decoder è addestrato a ricostruire le strutture coerenti dell'immagine. In questo modo, l'output risulta spesso più regolare e meno rumoroso rispetto all'input.

Esistono due principali approcci all'uso degli autoencoder per il denoising:

1. **Denoising Autoencoder (DAE)** – addestrati esplicitamente fornendo in input immagini rumorose e in output le corrispondenti immagini pulite;
2. **Variational Autoencoder (VAE)** – che sfruttano una formulazione probabilistica per modellare la distribuzione delle immagini, con la possibilità di generare nuove istanze coerenti e prive di rumore.

Sebbene le prestazioni di base degli autoencoder siano inferiori rispetto a modelli più specializzati come RIDNet o DnCNN, essi rappresentano un approccio estremamente flessibile e didatticamente utile, oltre a costituire la base di architetture moderne più complesse (ad esempio le GAN e le reti transformer-based per il denoising).

2.3.1 Implementazione Autoencoder

Definizione e primo livello convoluzionale e pooling

Listing 2.19: Definizione e primo livello convoluzionale

```

1 def build_autoencoder(input_shape):
2     input = Input(input_shape, name = 'Input')
3     x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
      ↪ he_normal',
4         activation = 'relu', padding = 'same', name = '
      ↪ Conv2d_1')(input)
5     x = MaxPool2D(name = 'Maxpool_1')(x)

```

La funzione `build_autoencoder` riceve come parametro la dimensione delle immagini di input. Il layer di input è il punto da cui i dati vengono forniti alla rete.

Si applica una convoluzione con 64 filtri 3×3 , con inizializzazione dei pesi `he_normal` e funzione di attivazione `ReLU`.

Segue un'operazione di **max pooling**, che riduce la dimensione spaziale (compressione), mantenendo le informazioni più rilevanti.

Secondo livello di convoluzione e pooling

Listing 2.20: Secondo livello convolutivo + pooling

```

1 x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
2     activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_2')(x)
3 x = MaxPool2D(name = 'Maxpool_2')(x)

```

Viene applicata una nuova convoluzione con le stesse caratteristiche della precedente, seguita da un ulteriore **max pooling**. Questa sequenza riduce ulteriormente le dimensioni spaziali e costituisce la parte di **encoder**, ossia compressione delle informazioni.

Inizio del decoder con upsampling

Listing 2.21: Terzo livello convolutivo + upsampling

```

1 x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
2     activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_3')(x)
3 x = UpSampling2D(name = 'Upsample_1')(x)
4 x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
5     activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_4')(x)
6 x = UpSampling2D(name = 'Upsample_2')(x)

```

Dopo la compressione, inizia la fase di ricostruzione (**decoder**).

Viene applicata una convoluzione seguita da un'operazione di **upsampling**, che aumenta la dimensione spaziale riportandola gradualmente alla risoluzione originale con una convoluzione a 64 filtri seguita da un upsampling, che ricostruisce ulteriormente l'immagine.

Strato finale

Listing 2.22: Strato finale dell'autoencoder

```

1 x = Conv2D(3, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
2     activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_5')(x)
3
4 return Model(input, x)

```

Output della rete Infine, uno strato convoluzionale con 3 filtri (uno per canale RGB) ricostruisce l'immagine di output. La funzione `Model` collega l'input con l'output, restituendo il modello completo.

3 Dataset Utilizzati

L'efficacia di qualunque modello di visione artificiale dipende in larga misura dalla qualità e dalla varietà dei dati su cui viene addestrato. Per questo motivo, sono stati utilizzati cinque dataset pubblici largamente adottati nella comunità scientifica. I dataset scelti sono divisi in due gruppi: il primo composto da tre dataset BSD500(Martin et al. 2001), DIV2K(Ignatov, Timofte, et al. 2019) e MIT-Adobe FiveK(Bychkovsky et al. 2011) a cui è stato applicato un rumore artificiale, mentre il secondo gruppo sono due dataset SSID(Abdelhamed, Timofte, Brown, et al. 2019) e RENOIR(Anaya and Barbu 2014) con rumore reale

Nello specifico, i dataset sono:

- **BSD500** – per valutazioni di segmentazione e denoising su immagini naturali;
- **DIV2K** – per il task di super-risoluzione;
- **MIT-Adobe FiveK** – per il miglioramento automatico delle immagini;
- **SSID** – per il denoising su immagini reali catturate da smartphone;
- **RENOIR** – per la rimozione di rumore in scenari reali e a bassa illuminazione.

3.1 BSD500

Il **Berkeley Segmentation Dataset 500** (BSD500) è un benchmark classico nell'ambito della visione artificiale, introdotto originariamente per valutare algoritmi di segmentazione semantica. Il dataset contiene un totale di 500 immagini naturali ad alta qualità, suddivise in tre insiemi: 200 per l'addestramento, 100 per la validazione e 200 per il test.

Ogni immagine è accompagnata da annotazioni di segmentazione eseguite manualmente da più utenti, fornendo così un confronto tra segmentazioni automatiche e percezione umana.

Nonostante il suo scopo originale non fosse legato alla rimozione del rumore o alla super-risoluzione, la sua varietà contenutistica e la buona qualità visiva ne hanno determinato un uso esteso in numerosi lavori di *image denoising*, *super-resolution* e compressione.

Caratteristiche principali:

- Immagini naturali di alta qualità;
- Diversità semantica (scene urbane, naturali, indoor/outdoor);
- Annotazioni di segmentazione manuali;
- Esteso utilizzo in compiti di visione oltre la segmentazione.

3.2 DIV2K

Il dataset **DIV2K** (DIVERse 2K resolution images) è uno dei benchmark più utilizzati per la valutazione di algoritmi di super-risoluzione. Contiene 1000 immagini ad alta definizione (risoluzione 2K), delle quali:

- 800 sono dedicate al training,
- 100 alla validazione,
- 100 al test.

Le immagini sono state selezionate per garantire diversità semantica e alta qualità visiva, provenendo da diverse fonti fotografiche. Il dataset include versioni degradate delle immagini originali (LR – Low Resolution) ottenute tramite downscaling con vari metodi, tra cui bicubico, bicubico + blur e compressione JPEG, consentendo lo sviluppo e la comparazione di metodi avanzati di upscaling.

Caratteristiche principali:

- Immagini ad alta risoluzione (2K);
- Disponibilità di versioni degradate a più scale ($\times 2$, $\times 3$, $\times 4$);
- Uso esteso in challenge internazionali (NTIRE);
- Elevata diversità semantica.

3.3 MIT-Adobe FiveK

Il **MIT-Adobe FiveK** è un dataset pensato per il miglioramento delle immagini (*image enhancement*). È composto da 5000 fotografie scattate in formato RAW da vari fotografi, e ogni immagine è accompagnata da cinque versioni modificate da esperti di fotoritocco, fornendo un riferimento multiplo per la qualità “ideale” di un’immagine.

Il dataset è particolarmente adatto per addestrare modelli che mirano a replicare interventi di editing umano (esposizione, contrasto, bilanciamento del bianco, saturazione), rendendolo un riferimento di alta qualità nei task di fotoritocco automatico e personalizzazione dello stile.

Caratteristiche principali:

- Immagini RAW e versioni ritoccate da 5 esperti;
- Varietà di scene (ritratti, paesaggi, interni, notturne);
- Impiego in modelli di apprendimento supervisionato per image enhancement;
- Disponibilità di dati in formato ad alta profondità di bit.

3.4 SSID

Il **Smartphone Image Denoising Dataset** (SSID) è stato sviluppato per colmare il divario tra denoising su immagini sintetiche e denoising in condizioni reali. Le immagini contenute nel dataset sono state acquisite con dispositivi mobili e presentano rumore autentico generato in condizioni di bassa luminosità.

Ogni immagine “noisy” è accompagnata dalla relativa versione “clean”, ottenuta mediante esposizioni multiple o tecniche di denoising hardware, e le immagini sono state acquisite in vari ambienti indoor e outdoor.

Caratteristiche principali:

- Acquisizione da smartphone in condizioni reali;
- Coppie noisy-clean disponibili;
- Diversi livelli di ISO e condizioni di luce;
- Utile per sviluppare modelli robusti al rumore reale.

3.5 RENOIR

Il **RENOIR** (Real Noise Image Dataset) è un dataset specificamente progettato per studiare la rimozione del rumore reale presente in immagini fotografiche acquisite in condizioni difficili. Le immagini sono state ottenute usando DSLR e smartphone, e ogni scena è catturata più volte: con esposizione breve (noisy) e lunga (clean), fornendo così una ground truth affidabile.

Una delle caratteristiche distintive di RENOIR è la presenza di rumore proveniente da varie fonti: termico, elettronico, ISO elevato e compressione, tutti elementi che pongono sfide reali nel pre-processing delle immagini.

Caratteristiche principali:

- Immagini reali con rumore (no synthetic noise);
- Acquisite con più fotocamere e in diverse condizioni;
- Triple noisy-clean-reference;
- Utile per validare l'efficacia di algoritmi di denoising realistici.

3.6 Kvasir dataset

Idataset impiegato per effettuare fine tuning è il dataset Kvasir (**Pogorelov:2017:KMI:3083187.3083212**) il quale comprende 1.000 immagini endoscopiche ad alta risoluzione, in particolare raffiguranti polipi e altre anomalie del tratto gastrointestinale. Sviluppato per supportare la ricerca nel campo della diagnosi assistita da computer (CADx), con l'obiettivo di migliorare l'accuratezza e la tempestività dell'identificazione di patologie gastrointestinali.

4 Elaborazione dei Dataset

In questa sezione saranno inseriti i dettagli tecnici relativi alla fase di preprocessing ed elaborazione dei dataset sopra descritti. I punti che verranno sviluppati includono:

I vari dataset sono stati però manipolati prima di passarli a SOGGETTP usando come riferimento l'esperimento di Real image denoising with feature attention (Anwar and Barnes 2019) sono state estratte 12 patch da 512x512 dai vari dataset con immagini 4k, mentre da BSD500 che ha immagini più piccole è stato effettuato un padding con specchiamento dei bordi per arrivare sempre ad una risoluzione di 512x512

Il preprocessing delle immagini è un passaggio cruciale per la standardizzazione del dataset, soprattutto in contesti in cui le dimensioni delle immagini devono essere uniformi per essere elaborate da modelli convoluzionali. Le tre modalità implementate coprono i principali scenari applicativi, permettendo flessibilità tra conservazione del contenuto e adattamento geometrico.

- Conversione di immagini RAW in RGB jpg (per MIT-Adobe FiveK);
- Suddivisione in patch da 512x512
- padding fino a 512x512
- Normalizzazione dei valori dei pixel (scaling da $[0, 255]$ a $[0, 1]$ o $[-1, 1]$); i-quando carico i dataset
- Strategie per la gestione di coppie noisy-clean

Questa parte è fondamentale per garantire che i dati in ingresso ai modelli siano coerenti, bilanciati e in grado di migliorare la convergenza durante la fase di addestramento.

4.0.1 Conversione immagini RAW

Le immagini contenute nei dataset come MIT-Adobe FiveK o RENOIR sono spesso fornite in formato RAW. Per utilizzare tali immagini nei pipeline di addestramento dei modelli di deep learning, è necessario convertirle in un formato più accessibile e standard.

A tal fine, è stato sviluppato uno script Python che sfrutta le librerie `rawpy` e `imageio` per effettuare la conversione in maniera ricorsiva su intere cartelle. Di seguito viene analizzato il codice sorgente, commentandolo nei suoi blocchi funzionali.

Importazione dei Moduli e Definizione della Funzione

Listing 4.1: Conversione immagini raw a jpg

```

1 def convert_raw_to_jpg_recursive(input_folder, output_folder):
2     input_path = Path(input_folder)
3     output_path = Path(output_folder)
4     output_path.mkdir(parents=True, exist_ok=True)

```

In questo primo blocco:

- Viene definita una funzione `convert_raw_to_jpg_recursive` che prende in input il percorso della cartella contenente i file RAW e quello della cartella di destinazione;
- I percorsi vengono convertiti in oggetti `Path` della libreria `pathlib`, per semplificare la gestione cross-platform dei file system;
- Se la cartella di output non esiste, viene creata automaticamente tramite `mkdir(parents=True, exist_ok=True)`.

Estensioni RAW supportate

Listing 4.2: Estensioni immagini raw

```

1 raw_extensions = ['.CR2', '.NEF', '.ARW', '.RW2', '.RAF', '.
    ↪ DNG', '.ORF', '.PEF']

```

Si definisce una lista delle estensioni più comuni per i file RAW prodotti da vari modelli di fotocamere digitali. Queste estensioni verranno utilizzate per filtrare i file da convertire.

Scansione Ricorsiva della Cartella

Listing 4.3: Scansione ricorsiva cartelle

```

1 for raw_file in input_path.rglob('*'):
2     if raw_file.suffix.upper() in raw_extensions and raw_file
    ↪ .is_file():

```

Qui si esegue una scansione ricorsiva all'interno della cartella di input usando `rglob('*')`, che restituisce tutti i file e sottocartelle. Per ogni file:

- Si verifica che l'estensione (in maiuscolo) appartenga alla lista di estensioni RAW supportate;
- Si controlla che l'elemento sia effettivamente un file (e non una directory).

Lettura e Conversione del File RAW

Listing 4.4: Lettura immagini con OpenCV

```

1     try:
2         with rawpy.imread(str(raw_file)) as raw:
3             rgb = raw.postprocess()

```

All'interno del blocco `try`, viene utilizzata la libreria `rawpy`, che fornisce un'interfaccia per la decodifica dei file RAW. La funzione `postprocess()` converte i dati RAW nel formato RGB, eseguendo demosaicizzazione, bilanciamento del bianco e correzione del colore.

Generazione del Nome File di Output

Listing 4.5: Rinomina file

```

1      parent_name = raw_file.parent.name
2      output_filename = f {parent_name}_{raw_file.
    ↪ stem}.jpg
3      output_file = output_path / output_filename

```

Per generare un nome univoco al file JPG risultante:

- Viene prelevato il nome della cartella contenente il file RAW (utile per mantenere traccia della struttura);
- Il nome del file finale è composto da <nome_cartella>_<nome_file>.jpg;
- Il file verrà salvato nella directory di output.

Scrittura del File e gestione delle eccezioni

Listing 4.6: Salvataggio immagine

```

1      imageio.imwrite(str(output_file), rgb)
2      print(f Convertito: {raw_file} -> {
    ↪ output_file.name} )
3  except Exception as e:
4      print(f Errore nella conversione di {raw_file.
    ↪ name}: {e} )

```

La libreria `imageio` viene utilizzata per salvare l'immagine RGB come file JPG. Viene anche stampato a console un messaggio di conferma con il nome del file convertito.

Infine, viene catturata qualsiasi eccezione durante il processo di conversione. Questo è utile per non interrompere lo script in caso di file corrotti o non leggibili. L'errore viene riportato a console per un'eventuale analisi successiva.

Questo script permette di gestire automaticamente grandi volumi di immagini RAW provenienti da più sorgenti e formati. La conversione a JPG è un passaggio fondamentale per:

- Uniformare i dati in ingresso ai modelli;
- Ridurre lo spazio occupato;
- Permettere l'applicazione di tecniche di preprocessing standard;
- Consentire la visualizzazione rapida dei dati convertiti.

L'esecuzione ricorsiva rende il sistema scalabile per dataset organizzati in sottocartelle, come spesso accade in raccolte fotografiche professionali.

4.0.2 Classificazione immagini GT-Noisy

Il dataset RENOIR contiene, per ogni scena, una serie di 4/5 immagini acquisite con diversi tempi di esposizione. Questa caratteristica introduce un rumore naturale nelle acquisizioni, rendendo necessario distinguere automaticamente le immagini di riferimento (*ground truth*, GT) da quelle rumorose (*noisy*).

Per questa operazione sono stati utilizzati due script principali: uno per classificare le immagini e uno per duplicare l'immagine GT in corrispondenza di ciascuna immagine rumorosa. Di seguito viene illustrato il codice di classificazione e il suo funzionamento.

Listing 4.7: Caricamento delle immagini di RENOIR

```

1 def load_images_by_batch(input_folder):
2     batches = defaultdict(list)
3     for fname in os.listdir(input_folder):
4         if fname.lower().endswith( '.jpg' ) and fname.startswith(
5             ↪ Batch_ ):
6             parts = fname.split( _ , 2)
7             if len(parts) >= 3:
8                 batch_id = parts[1]
9                 batches[batch_id].append(fname)
10    return batches

```

La funzione `load_images_by_batch` organizza le immagini in gruppi, basandosi sul prefisso del nome file (`Batch_XXX`). In questo modo, ogni batch di immagini viene identificato in maniera univoca ed è possibile elaborarlo separatamente.

Listing 4.8: Mean Squared Error

```

1 def mse(img1, img2):
2     err = np.mean((img1.astype( float32 ) - img2.astype( float32
3         ↪ )) ** 2)
4     return err

```

La funzione `mse` calcola il *Mean Squared Error*, ovvero la differenza media al quadrato tra due immagini.

Questo indice è molto usato per stimare quanto un'immagine differisca da un'altra: più il valore è basso, maggiore è la somiglianza.

Listing 4.9: Classificazione batch con MSE

```

1 def classify_batch(batch_images, input_folder):
2     loaded = {}
3     for fname in batch_images:
4         path = os.path.join(input_folder, fname)
5         img = cv2.imread(path)
6         if img is not None:
7             img_resized = cv2.resize(img, (256, 256))
8             loaded[fname] = img_resized
9         else:
10            print(f [!] Immagine non caricata: {fname} )

```

La funzione `classify_batch` carica tutte le immagini del batch e le ridimensiona ad una dimensione fissa (256×256).

Questo passaggio assicura coerenza nelle dimensioni, fondamentale per il calcolo corretto della similarità.

Listing 4.10: Calcolo di MSE

```

1  mse_scores = {}
2  for name1, img1 in loaded.items():
3      total_error = 0
4      for name2, img2 in loaded.items():
5          if name1 != name2:
6              total_error += mse(img1, img2)
7      avg_error = total_error / (len(loaded) - 1)
8      mse_scores[name1] = avg_error
9
10 gt_image = min(mse_scores, key=mse_scores.get)

```

Successivamente, per ogni immagine del batch si calcola l'errore medio rispetto a tutte le altre (escludendo i confronti con se stessa). L'immagine con il valore medio di MSE più basso viene identificata come *ground truth*, poiché è quella che differisce meno dalle altre.

Listing 4.11: Elaborazione dei risultati

```

1  result = {}
2  for fname in batch_images:
3      label = GT if fname == gt_image else NOISE
4      result[fname] = (label, mse_scores.get(fname, -1)) # -1
5                      ↪ se immagine non caricata
6  return result

```

In questa fase, i risultati vengono raccolti in un dizionario: l'immagine riconosciuta come GT riceve l'etichetta GT, mentre le altre vengono etichettate come NOISE. Per ciascun file viene salvato anche il relativo punteggio di errore.

Listing 4.12: Salvataggio della classificazione

```

1  def save_classification(result_dict, output_file):
2      MSE_MAX = 255.0
3      with open(output_file, 'w') as f:
4          for fname, (label, score) in result_dict.items():
5              f.write(f '{fname} -> {label} [MSE: {score:.2f} -
                      ↪ ERROR: {(score/MSE_MAX):.2f}]\n ')

```

La funzione `save_classification` scrive i risultati su un file di testo, indicando per ciascuna immagine: il nome del file, la classificazione (GT/NOISE) e il relativo valore di errore normalizzato rispetto al massimo MSE possibile.

Listing 4.13: Funzione principale per la suddivisione e classificazione in batch

```

1 def main(input_folder, output_file):
2     batches = load_images_by_batch(input_folder)
3     all_results = {}
4     for batch_id, images in batches.items():
5         if len(images) < 2:
6             print(f [!] Batch {batch_id} ha meno di 2 immagini,
7                 ↪ ignorato. )
8             continue
9
10            print(f [+] Elaborazione Batch {batch_id} ({len(images)}
11                ↪ immagini) )
12            batch_result = classify_batch(images, input_folder)
13            all_results.update(batch_result)
14            save_classification(all_results, output_file)
15            print(f Classificazione completata. Risultati salvati in: {
16                ↪ output_file} )

```

Infine, la funzione `main` coordina l'intero processo:

1. suddivide le immagini in batch,
2. applica la classificazione GT/NOISE ad ogni gruppo,
3. raccoglie i risultati complessivi,
4. e li salva su file.

Questo garantisce una pipeline automatica e ripetibile per la classificazione delle immagini del dataset RENOIR.

Partendo dal caricamento delle batch di immagini con `load_images_by_batch`^{4.7}, vengono ignorate le batch con 2 immagini e si procede con la classificazione con `classify_batch`^{4.9} e l'aggiunta dei risultati ai risultati totali che sono poi scritti su file attraverso `save_classification`^{4.12}

4.0.3 Duplicazione immagini

Una volta classificata l'immagine come *ground truth* (gt), essa viene duplicata e associata per ognuna di quelle classificate come *noisy*. Il seguente script Python automatizza il processo, organizzando le immagini e garantendo la coerenza della nomenclatura.

Listing 4.14: Definizione della funzione per processare la directory

```

1 def process_batch_folder(batch_folder, parent_dir):
2     batch_name = os.path.basename(batch_folder)
3     print(f [+] Processing {batch_name} )
4     copied_files = []

```

La funzione `process_batch_folder` riceve in ingresso una cartella di batch e la cartella principale. Inizia recuperando il nome del batch e prepara una lista per memorizzare i file duplicati o rinominati.

Listing 4.15: Step 1: Gestione Reference

```

1  reference_files = glob.glob(os.path.join(batch_folder, '*'
      ↳ Reference.bmp'))
2  if not reference_files:
3      print(f Nessun file Reference trovato in {batch_name} )
4  else:
5      ref = reference_files[0]
6      base = os.path.splitext(os.path.basename(ref))[0]
7      dest1_name = f {base}_1.bmp
8      dest2_name = f {base}_2.bmp
9      dest1 = os.path.join(batch_folder, dest1_name)
10     dest2 = os.path.join(batch_folder, dest2_name)
11     shutil.copy(ref, dest1)
12     shutil.copy(ref, dest2)
13     os.remove(ref)
14     copied_files += [dest1, dest2]

```

Nel Primo Step si gestiscono le immagini di riferimento (Reference). Se presenti, vengono duplicate e rinominate con suffissi numerici (_1, _2). L'originale viene poi eliminato, così da mantenere solo i duplicati coerenti.

Listing 4.16: Step 2: Gestione Noisy

```

1  noisy_files = sorted(glob.glob(os.path.join(batch_folder, '*'
      ↳ Noisy.bmp'))))
2  new_noisy_paths = []
3  for idx, path in enumerate(noisy_files, start=1):
4      base = os.path.splitext(os.path.basename(path))[0]
5      new_name = f {base}_{idx}.bmp
6      new_path = os.path.join(batch_folder, new_name)
7      os.rename(path, new_path)
8      copied_files.append(new_path)

```

Nel Secondo Step vengono gestite le immagini rumorose (Noisy). Ognuna viene rinominata in modo incrementale, aggiungendo un indice alla fine del nome, così da distinguerle chiaramente e mantenerle ordinate.

Listing 4.17: Step 3: Aggiunta del prefisso Batch_XXX_

```

1  renamed_paths = []
2  for fname in os.listdir(batch_folder):
3      if fname.endswith( '.bmp' ) and not fname.startswith(
4          ↪ batch_name + '_' ):
5          old_path = os.path.join(batch_folder, fname)
6          new_name = f '{batch_name}_{fname}'
7          new_path = os.path.join(batch_folder, new_name)
8          os.rename(old_path, new_path)
9          # Aggiorna anche la lista dei file da copiare
10         if old_path in copied_files:
11             renamed_paths.append(new_path)
12     elif fname.endswith( '.bmp' ) and fname.startswith(
13         ↪ batch_name + '_' ):
14         renamed_paths.append(os.path.join(batch_folder, fname
15             ↪ ))

```

Il Terzo Step si occupa di uniformare i nomi dei file aggiungendo il prefisso Batch_XXX_, dove XXX è il nome della cartella batch. In questo modo, tutte le immagini risultano facilmente tracciabili rispetto al batch di origine.

Listing 4.18: Step 4: Copia nella cartella principale

```

1  for file_path in renamed_paths:
2      dest_path = os.path.join(parent_dir, os.path.basename(
3          ↪ file_path))
4      shutil.copy(file_path, dest_path)
5      print(f Copiato {os.path.basename(file_path)} -> {
6          ↪ parent_dir} )

```

Nel Quarto Step le immagini rinominate vengono copiate nella cartella principale, così da raccogliere i dati elaborati da più batch in un unico luogo centralizzato.

Listing 4.19: Utilizzo della funzione process_batch_folder

```

1  def main(parent_dir):
2      for folder in sorted(os.listdir(parent_dir)):
3          full_path = os.path.join(parent_dir, folder)
4          if folder.startswith( Batch_ ) and os.path.isdir(
5              ↪ full_path):
6              process_batch_folder(full_path, parent_dir)

```

La funzione main scorre tutte le cartelle nella directory principale, seleziona quelle che iniziano con Batch_ e applica la funzione process_batch_folder per elaborarle.

4.0.4 Padding di immagini con risoluzione inferiore a 512x512

Nel contesto dell'addestramento dei modelli di deep learning, è spesso necessario uniformare la dimensione delle immagini in input. Questo script Python permette di adattare immagini a una risoluzione di 512×512 pixel.

Definizione delle Costanti

Il dataset BSD 500 è composto da tre cartelle con immagini non RAW e dimensioni variabili inferiori alla risoluzione 512x512

Listing 4.20: Costanti con directry per BSD

```
1 TRAIN_512_DIR = BSD_512/train
2 TEST_512_DIR = BSD_512/test
3 VAL_512_DIR = BSD_512/val
4
5 DIR_TEST = testdir
6 DIR_RESULT = resultdir
```

Sono definite le directory di input e output, rispettivamente per i set di training, validation e test.

Padding centrato

Listing 4.21: Funzione di padding

```
1 def pad_to_512(img):
2     h, w = img.shape[:2]
3     pad_bottom = max(0, 512 - h)
4     pad_right = max(0, 512 - w)
```

Viene presa l'immagine e tramite `img.shape[:2]` e si estraggono altezza(h) e larghezza(w). Poi si calcola per le 2 grandezze lo scarto di pixel che manca ad arrivare a 512

Listing 4.22: Creazione del padding sui lati dell'immagine

```
1     pad_b_1 = pad_bottom // 2
2     pad_b_2 = pad_bottom - pad_b_1
3     pad_r_1 = pad_right // 2
4     pad_r_2 = pad_right - pad_r_1
5     return cv2.copyMakeBorder(img, pad_b_1, pad_b_2, pad_r_1,
        ↪ pad_r_2, cv2.BORDER_REFLECT_101)
```

Poi gli scarti calcolati vengono suddivisi per mantenere il padding centrato.

Infine viene applicato un bordo riflesso(padding) tramite `cv2.copyMakeBorder` su tutti lati dell'immagine mantenendo la risoluzione originale, dunque senza snaturarla e aumentandone le dimensioni a quelle volute.

`cv2.BORDER_REFLECT_101` è una costante di OpenCV utilizzata per specificare se l'immagine vada specchiata o ripetuta sui bordi riflessi

Funzione di elaborazione batch delle immagini

Listing 4.23: Definizione funzione di elaborazione immagini

```
1 def process_images(input_dir, output_dir):
2     count = 0
3     for fname in os.listdir(input_dir):
4         if not fname.lower().endswith(( '.png' , '.jpg' , '.jpeg' )):
5             continue
```

Ad ogni file viene controllata l'estensione, in caso non sia un'immagine il programma lo salta, non precludendo l'esecuzione agli altri file

```
1         img_path = os.path.join(input_dir, fname)
2         img = cv2.imread(img_path)
3         if img is None:
4             print(f [!] Errore nel leggere {fname} )
5             continue
6
7         out_img = pad_to_512(img)
8
9         out_path = os.path.join(output_dir, fname)
10        cv2.imwrite(out_path, out_img)
11        count += 1
12
13    print(f {count} immagini processate )
```

Viene fatto un'unione di percorso e nome immagine per compatibilità di sistema e convertita in un oggetto manipolabile da OpenCV.

Controlla che l'immagine non sia corrotta o illeggibile, applica il padding e crea il file nella cartella di output. Alla fine stampa il numero di file processati

4.1 Patching

Il **patching** è una tecnica comunemente utilizzata nell'ambito dell'elaborazione di immagini e dell'addestramento di reti neurali convoluzionali, che consiste nel suddividere un'immagine di grandi dimensioni in più sotto-immagini quadrate o rettangolari, dette *patch*. Questo approccio presenta diversi vantaggi: riduce il consumo di memoria e il tempo di addestramento, permette di aumentare la quantità di dati disponibili (poiché da una singola immagine possono essere ricavati numerosi patch), e consente al modello di concentrarsi su porzioni locali dell'immagine, migliorando la capacità di apprendere dettagli fini e strutture locali.

Durante la fase di ricostruzione o inferenza, i patch possono essere riuniti per ricomporre l'immagine originale, spesso con tecniche di sovrapposizione e fusione per evitare artefatti lungo i bordi. In sintesi, il patching rappresenta una strategia essenziale per la gestione efficiente di dataset ad alta risoluzione e per l'ottimizzazione delle prestazioni nei modelli di visione artificiale.

Listing 4.24: Costanti per il patching

```

1 PATCH_SIZE = 512
2 # Layout per immagini orizzontali
3 ROWS_HORIZONTAL = 3
4 COLS_HORIZONTAL = 4
5 # Layout per immagini verticali
6 ROWS_VERTICAL = COLS_HORIZONTAL
7 COLS_VERTICAL = ROWS_HORIZONTAL

```

Costanti usate per definire la dimensione delle patch e il loro numero totale tenendo conto che l'immagine sia verticale o orizzontale.

Listing 4.25: Padding per il patching

```

1 def pad_image(img, target_width, target_height):
2     h, w = img.shape[:2]
3     pad_bottom = max(0, target_height - h)
4     pad_right = max(0, target_width - w)
5     padded = cv2.copyMakeBorder(img, 0, pad_bottom, 0, pad_right,
6     ↪ cv2.BORDER_REFLECT_101)
7     return padded

```

Fare padding su immagini grandi per renderle divisibili per la dimensione delle patch, delle colonne e delle righe.

In questo caso vengono aggiunti pixel solo nei lati **BOTTOM** e **RIGHT**.

Listing 4.26: Funzione di estrazione patch dalle immagini

```

1 def extract_orientation_aware_patches(image, patch_size=
2     ↪ PATCH_SIZE):
3     h, w = image.shape[:2]
4     if w >= h:
5         rows, cols = ROWS_HORIZONTAL, COLS_HORIZONTAL
6     else:
7         rows, cols = ROWS_VERTICAL, COLS_VERTICAL
8     total_height = patch_size * rows
9     total_width = patch_size * cols
10
11     img = pad_image(image, total_width, total_height)

```

Questa funzione parte prendendo altezza e larghezza dell'immagine passata, controlla se deve usare le costanti per immagini orizzontali o verticali, calcola le dimensioni per il padding e poi

Listing 4.27: Calcolo dei passi e creazione patch

```

1  row_step = (h - patch_size) // (rows - 1) if rows > 1 else 0
2  col_step = (w - patch_size) // (cols - 1) if cols > 1 else 0
3  patches = []
4  for i in range(rows):
5      for j in range(cols):
6          y = i * row_step
7          x = j * col_step
8          patch = img[y:y+patch_size, x:x+patch_size]
9          if patch.shape[0] == patch_size and patch.shape[1] ==
           ↪ patch_size:
10             patches.append(patch)
11
12  return patches

```

Questo frammento di codice si occupa di suddividere un'immagine in piccole sotto-immagini, chiamate *patches*, che vengono estratte in modo regolare lungo righe e colonne.

Per determinare la distanza fra una patch e la successiva, vengono calcolati `row_step` per il passo verticale e `col_step` per il passo orizzontale.

Successivamente, due cicli annidati percorrono l'immagine riga per riga e colonna per colonna, calcolando di volta in volta le coordinate `x` e `y` di partenza di ogni patch. Ogni ritaglio ha dimensione fissa (`patch_size`) e viene verificato che corrisponda esattamente alle dimensioni attese prima di essere aggiunto alla lista `patches`.

Infine, la funzione restituisce l'insieme di tutte le patch ottenute, che potranno essere usate come input per modelli di deep learning o per altre operazioni di elaborazione delle immagini.

```

1  def process_folder(input_dir, output_dir):
2      os.makedirs(output_dir, exist_ok=True)
3      supported_ext = ( .png , .jpg , .jpeg , .dng )
4      filenames = [f for f in os.listdir(input_dir) if f.lower().
           ↪ endswith(supported_ext)]

```

Questa funzione serve per caricare tutte le immagini e creare le patch su disco. Viene chiamata la funzione per la cartella di output che crea la directory in caso non esista, vengono specificate le estensioni accettate dal programma, e viene stilata una lista con tutti i file trovati

Listing 4.28: Ciclo di elaborazione delle immagini e verifica preliminare

```

1  for fname in filenames:
2      print(f Processing {fname}... )
3      img_path = os.path.join(input_dir, fname)
4      img = cv2.imread(img_path)
5      if img is None:
6          print(f [!] Immagine non valida: {fname} )
7          continue
8      if img.shape[0] < PATCH_SIZE or img.shape[1] < PATCH_SIZE
9          ↪ :
10         print(f [!] Immagine troppo piccola: {fname} ({img.
11             ↪ shape[1]}x{img.shape[0]}) )
12         continue
13     patches = extract_orientation_aware_patches(img)

```

Viene eseguito un ciclo su tutti i file contenuti in una lista di immagini (`filenames`), processandoli uno alla volta.

Per ciascun file viene costruito il percorso completo e l'immagine viene caricata con `cv2.imread`. Se l'immagine non è valida o non ha la dimensioni richiesta, il programma passa al file successivo, evitando così crash o elaborazioni inutili.

Solo le immagini valide e sufficientemente grandi vengono infine suddivise in patch tramite la funzione `extract_orientation_aware_patches`.

```

1      base_name = os.path.splitext(fname)[0]
2      for i, patch in enumerate(patches):
3          out_name = f {base_name}_patch{i}.jpg
4          cv2.imwrite(os.path.join(output_dir, out_name), patch
5              ↪ )
6      print(f Patch salvate in: {output_dir} )

```

Per ogni patch creata questa avrà il nome dell'immagine originale seguita dal numero di patch corrispondente per poi essere scritta su disco usando sempre OpenCV.

4.2 Aggiunta del rumore

Listing 4.29: Aggiunta di rumore da lista

```

1  def add_noise(x):
2      noise_levels = [15 / 255.0, 25 / 255.0, 50 / 255.0]
3      noise_factor = tf.random.shuffle(noise_levels)[0]
4      noise = tf.random.normal(shape=tf.shape(x), mean=0.0, stddev=
5          ↪ noise_factor)
6      x_noisy = x + noise
7      x_noisy = tf.clip_by_value(x_noisy, 0.0, 1.0)
8      return x_noisy

```

Listing 4.30: Aggiunta del rumore da funzione

```

1  def add_noise_2(x, noise_factor):
2      noise = tf.random.normal(shape=tf.shape(x), mean=0.0, stddev=
3          ↪ noise_factor)

```

```
3     x_noisy = x + noise
4     x_noisy = tf.clip_by_value(x_noisy, 0.0, 1.0)
5     return x_noisy
```

5 Analisi ed ottimizzazione

5.1 Prestazioni

PSNR è il metodo più comune per misurare la qualità delle immagini.

Il PSNR è definito come il rapporto tra il massimo valore possibile di un segnale e il valore del rumore che disturba la qualità della sua rappresentazione.

Normalmente misurato in una scala logaritmica in decibel(dB).

Data l'immagine originale(g) e l'immagine rumorosa(f), il PSNR è definito come:

$$PSNR = 20 \log_{10} \left(\frac{MAX_f}{\sqrt{MSE}} \right)$$

dove MAX_f è il valore massimo del pixel dell'immagine e si calcola come

$$MSE = \frac{1}{mn} \sum_0^{m-1} \sum_0^{n-1} \|f(i, j) - g(i, j)\|^2$$

mentre MSE (*Mean Square Error*) è l'errore quadratico medio tra l'immagine originale e quella rumorosa.

5.2 Prima run

5.3 Quantizzazione dei modelli

5.3.1 Performance su dataset sconosciuti

6 Fine Tuning

7 Study

Bibliography

- [1] James R. Janesick. *Photon Transfer: $DN \rightarrow \lambda$* . Bellingham, WA, USA: SPIE Press, 2007. ISBN: 978-0-8194-6722-5. URL: <https://picture.iczhiku.com/resource/eetop/shITqHhziPtOLvcx.pdf>.
- [2] Alessandro Foi et al. “Practical Poissonian–Gaussian Noise Modeling and Fitting for Single-Image Raw-Data”. In: *IEEE Transactions on Image Processing* 17.10 (2008), pp. 1737–1754. URL: https://webpages.tuni.fi/foi/papers/Foi-PoissonianGaussianClippedFitting-2007-IEEE_TIP.pdf.
- [3] Markku Mäkitalo and Alessandro Foi. “Optimal inversion of the generalized Anscombe transformation for Poisson–Gaussian noise”. In: *IEEE Transactions on Image Processing* 22.1 (2013), pp. 91–103. DOI: 10.1109/TIP.2012.2202675. URL: <https://webpages.tuni.fi/foi/papers/OptGenAnscombeInverse-doublecolumn-preprint.pdf>.
- [4] Hilario Gómez-Moreno et al. “A “Salt and Pepper” Noise Reduction Scheme for Digital Images Based on Support Vector Machines Classification and Regression”. In: *The Scientific World Journal* 2014 (2014), Article ID 826405. DOI: 10.1155/2014/826405. URL: <https://www.hindawi.com/journals/tswj/2014/826405/>.
- [5] J. W. Goodman. “Some fundamental properties of speckle”. In: *Journal of the Optical Society of America* (1976). seminal treatment of speckle statistics and properties; vedi anche review papers su despeckling per applicazioni medicali e SAR. URL: <https://materias.df.uba.ar/15a2021c1/files/2021/05/goodman1976.pdf>.
- [6] G. K. Wallace. “The JPEG still-picture compression standard”. In: *Communications of the ACM* 34.4 (1991), pp. 30–44. URL: https://www.lirmm.fr/~wpuech/enseignement/master_informatique/Compression_Insertion/articles/15_Wallace_JPEG_1992.pdf.
- [7] R. Eschbach. “Conditional Post-Processing of JPEG Compressed Images”. In: *Proceedings (paper/report)* (1997). vedi anche letteratura su riduzione artefatti JPEG e blocking artifacts. URL: <https://www.imaging.org/common/uploaded%20files/pdfs/Papers/1997/IST-0-4/122.pdf>.
- [8] Ery Arias-Castro and David L. Donoho. “Does median filtering truly preserve edges better than linear filtering?” In: *Annals of Statistics* (2009). analisi comparativa tra filtro mediano e filtro lineare.
- [9] Carlo Tomasi and Roberto Manduchi. “Bilateral Filtering for Gray and Color Images”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 1998, pp. 839–846.
- [10] Sylvain Paris et al. “Bilateral Filtering: Theory and Applications”. In: *Foundations and Trends® in Computer Graphics and Vision* 4.1 (2008), pp. 1–73.
- [11] David L. Donoho. “De-noising by soft thresholding”. In: *IEEE Transactions on Information Theory* 41.3 (1995), pp. 613–627.

- [12] Antonin Chambolle et al. “A Parameter Selection Method for Wavelet Shrinkage Denoising”. In: *BIT Numerical Mathematics* 43 (2003), pp. 611–626.
- [13] Anonymous Unsupervised Wavelet Denoising. “A wavelet denoising approach based on unsupervised learning model”. In: *EURASIP Journal on Advances in Signal Processing* 2020 (2020).
- [14] Antoni Buades, Bartomeu Coll, and Jean-Michel Morel. “A non-local algorithm for image denoising”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Vol. 2. IEEE. 2005, pp. 60–65.
- [15] Kostadin Dabov et al. “Image denoising by sparse 3D transform-domain collaborative filtering”. In: *IEEE Transactions on Image Processing* 16.8 (2007), pp. 2080–2095.
- [16] Linwei Fan et al. “Brief review of image denoising techniques”. eng. In: *Visual computing for industry, biomedicine and art* 2.1 (2019), pp. 7–12. ISSN: 2524-4442.
- [17] Hua Wang et al. “A Review of Image Denoising Methods”. In: *Communications in Information and Systems* 20.4 (2020). Un’introduzione esaustiva che classifica metodi classici e moderni per il denoising delle immagini, pp. 461–480. URL: https://www.intlpress.com/site/pub/files/_fulltext/journals/cis/2020/0020/0004/CIS-2020-0020-0004-a004.pdf.
- [18] Marc Lebrun and Arthur Leclaire. “An Implementation and Detailed Analysis of the K-SVD Image Denoising Algorithm”. In: *Image Processing On Line* 2 (2012). <https://doi.org/10.5201/ipol.2012.11m-ksvd>, pp. 96–133.
- [19] Wikipedia contributors. *K-SVD — Wikipedia, the free encyclopedia*. accessed August 27, 2025, general description of K-SVD algorithm. 2025. URL: <https://en.wikipedia.org/wiki/K-SVD>.
- [20] Quan Xiao et al. “Image Denoising via K-SVD with Primal-Dual Active Set Algorithm (K-SVD.P)”. In: *WACV. improvement over classical K-SVD for high noise levels*. 2020. URL: https://openaccess.thecvf.com/content_WACV_2020/papers/Xiao_Image_denoising_via_K-SVD_with_primal-dual_active_set_algorithm_WACV_2020_paper.pdf.
- [21] Jiechao Sheng et al. “SRNet: Sparse representation-based network for image denoising”. In: *Digital Signal Processing* 130 (2022). Embedding of sparse coding framework into CNN using unrolled optimization and multi-scale residual blocks, p. 103702. DOI: 10.1016/j.dsp.2022.103702. URL: <https://www.sciencedirect.com/science/article/abs/pii/S1051200422003190>.
- [22] Cristina Garcia-Cardona and Brendt Wohlberg. “Convolutional Dictionary Learning: A Comparative Review and New Algorithms”. In: *IEEE Transactions on Computational Imaging* 4.3 (2018). Survey and new approaches for convolutional sparse dictionary learning, pp. 366–381. DOI: 10.1109/TCI.2018.2840334. URL: <https://arxiv.org/abs/1709.02893>.
- [23] Yali Zhang et al. “Image Denoising using Convolutional Sparse Coding Network with Dry Friction”. In: *Proceedings of the Asian Conference on Computer Vision (ACCV)*. 2022, pp. 2322–2336.
- [24] Christian Marius Lillelund, Henrik Bagger Jensen, and Christian Fischer Pedersen. “Cloud K-SVD for Image Denoising”. In: *SN Computer Science* 3 (2022). Dictionary learning across multiple nodes. Originally appeared on arXiv as 2303.00755 in 2023, p. 151. DOI: 10.1007/s42979-022-01042-y. URL: <https://doi.org/10.1007/s42979-022-01042-y>.

- [25] Kai Zhang, Wangmeng Zuo, Yunjin Chen, et al. “Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising”. In: *IEEE Transactions on Image Processing* 26.7 (2017), pp. 3142–3155.
- [26] Kai Zhang, Wangmeng Zuo, and Lei Zhang. “FFDNet: Toward a fast and flexible solution for CNN-based image denoising”. In: *IEEE Transactions on Image Processing* 27.9 (2018), pp. 4608–4622.
- [27] Jaakko Lehtinen et al. “Noise2Noise: Learning image restoration without clean data”. In: *Proceedings of the International Conference on Machine Learning*. 2018.
- [28] Saeed Anwar and Nick Barnes. “Real Image Denoising With Feature Attention”. In: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 3155–3164. DOI: 10.1109/ICCV.2019.00325.
- [29] Anil Hore and D. Ziou. “Image Quality Metrics: PSNR vs. SSIM”. In: *International Journal of Computer Vision* 21.1 (2010), pp. 87–112. DOI: 10.1007/s00138-010-0230-4.
- [30] Zhou Wang et al. “Image Quality Assessment: From Error Visibility to Structural Similarity”. In: *IEEE Transactions on Image Processing* 13.4 (2004), pp. 600–612. DOI: 10.1109/TIP.2003.819861.
- [31] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. 4th. Pearson, 2018.
- [32] G. J. Yang T. S. Huang and G. Y. Tang. “A Fast Two-Dimensional Median Filtering Algorithm”. In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 27.1 (1979), pp. 13–18.
- [33] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. 2nd. Prentice Hall, 1999.
- [34] D. Martin et al. “A Database of Human Segmented Natural Images and its Application to Evaluating Segmentation Algorithms and Measuring Ecological Statistics”. In: *Proc. 8th Int’l Conf. Computer Vision*. Vol. 2. July 2001, pp. 416–423.
- [35] Vladimir Bychkovsky et al. “Learning Photographic Global Tonal Adjustment with a Database of Input / Output Image Pairs”. In: *The Twenty-Fourth IEEE Conference on Computer Vision and Pattern Recognition*. 2011.
- [36] Abdelrahman Abdelhamed, Radu Timofte, Michael S. Brown, et al. “NTIRE 2019 Challenge on Real Image Denoising: Methods and Results”. In: *The IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. June 2019.
- [37] Andrey Ignatov, Radu Timofte, et al. “PIRM challenge on perceptual image enhancement on smartphones: report”. In: *European Conference on Computer Vision (ECCV) Workshops*. Jan. 2019.
- [38] Josue Anaya and Adrian Barbu. “RENOIR - A Dataset for Real Low-Light Noise Image Reduction”. In: *arXiv preprint arXiv:1409.8230* (2014).
- [39] M. Aharon, M. Elad, and A. Bruckstein. “K-SVD: An Algorithm for Designing Overcomplete Dictionaries for Sparse Representation”. In: *IEEE Transactions on Signal Processing* 54.11 (2006), pp. 4311–4322.
- [40] Christian M. Lillielund, Henrik B. Jensen, and Christian F. Pedersen. “Cloud K-SVD for Image Denoising”. In: *SN Computer Science* 3 (2022). Dictionary learning across multiple nodes. Originally on arXiv:2303.00755, p. 151. DOI: 10.1007/s42979-022-01042-y. URL: <https://doi.org/10.1007/s42979-022-01042-y>.

- [41] Hua Wang et al. “A Review of Image Denoising Methods”. In: *Communications in Information and Systems* 20.4 (2020), pp. 461–480. URL: https://www.intlpress.com/site/pub/files/_fulltext/journals/cis/2020/0020/0004/CIS-2020-0020-0004-a004.pdf.